CMSC426

Jia-Bin Huang

Iskander Lou

Visual Odometry Final Project Report

The objective of this project was to reconstruct and visualize the 3D trajectory of a camera installed in a car using a sequence of driving frames. Visual odometry is the process of estimating the movement of a camera by looking at how the image changes from one frame to the next. In this project, I used a sequence of images from a driving dataset to estimate how the camera moved over time. The main goal was to identify shared keypoints between consecutive frames, estimate the relationship between the frames, recover the camera's rotation and translation, and then use those values to reconstruct the camera's path.

The project followed several main steps. First, I computed the camera intrinsic matrix using the provided camera model. Then, I loaded and converted the images from Bayer format into color images. After that, I detected and matched keypoints between consecutive frames using SIFT and a FLANN-based matcher. I then used the matched keypoints to estimate the fundamental matrix, computed the essential matrix from it, recovered the rotation and translation parameters, and finally used those parameters to reconstruct the trajectory of the camera. These steps follow the main structure of the project instructions, which required estimating the motion between successive frames and then using those estimates to plot the camera center over time.

To begin, I extracted the camera's intrinsic parameters using the provided camera model files. These parameters included the focal lengths fx and fy, as well as the principal point coordinates cx and cy. I used these values to construct the intrinsic camera matrix K:

$$K = \begin{bmatrix} fx & 0 & cx \\ 0 & fy & cy \\ 0 & 0 & 1 \end{bmatrix}$$

The intrinsic matrix is important because it describes the internal properties of the camera. It tells us how points from the 3D camera coordinate system are projected onto the 2D image plane. In this project, the intrinsic matrix was especially important because it was used later to compute the essential matrix from the fundamental matrix. The fundamental matrix describes the relationship between two images in pixel coordinates, while the essential matrix describes the relationship between two calibrated camera views.

Next, I loaded the images from the Oxford driving dataset. The original images were stored in Bayer format, so they could not be used as normal color images right away. I converted them into color images using cv2.cvtColor with the cv2.COLOR_BayerGR2BGR flag. This step is called demosaicing. It reconstructs a color image from the raw Bayer pattern. After converting the images, I saved them into a separate folder so that the rest of the pipeline could use the prepared images. This also made the later steps cleaner because I did not have to repeatedly convert the images inside every part of the code.

After preparing the images, I tested the process on one pair of consecutive frames first. This helped me make sure the feature detection, feature matching, and matrix estimation steps were working before running the full sequence. For feature detection, I used SIFT. SIFT stands for Scale-Invariant Feature Transform. It detects important keypoints in an image and creates descriptors for those keypoints. These descriptors describe the visual pattern around each keypoint, which makes it possible to compare keypoints between two different frames.

SIFT was a good choice for this project because the camera is moving, so the same object or feature may appear slightly different from one frame to the next. The scale, angle, lighting, or position of the feature can change. SIFT is useful because it is designed to find keypoints that are still recognizable even when these changes happen. This is important for visual odometry because the whole pipeline depends on finding reliable correspondences between consecutive frames.

Once I had keypoints and descriptors from two frames, I used a FLANN-based matcher to match the descriptors. FLANN stands for Fast Library for Approximate Nearest Neighbors. It finds similar descriptors between two images more efficiently than comparing every possible pair manually. In this project, the matcher returned possible matches between keypoints in the first frame and keypoints in the second frame.

However, not every match is reliable. Some image features can look similar even if they are not actually the same point in the real world. To remove weaker matches, I used Lowe's ratio test. For each descriptor, the matcher found the two closest matches. I kept the match only if the best match was clearly better than the second-best match. In my code, I used a threshold of 0.8, meaning that the best match had to have a distance less than 0.8 times the distance of the second-best match. This helped remove ambiguous matches and keep only stronger correspondences.

After filtering the matches, I visualized them using cv2.drawMatches. This showed lines connecting matched keypoints between two consecutive frames. This was useful because it gave me a quick way to check whether the matching process looked reasonable. If the matches looked random, then the later steps would not produce a good result. The match visualization showed that the program was able to find many shared points between the two frames.

Figure 1. Keypoint matches between two consecutive frames using SIFT and FLANN-based matching.

Next, I used the matched keypoints to estimate the fundamental matrix. I extracted the 2D coordinates of the matched points from both images and passed them into cv2.findFundamentalMat. The fundamental matrix describes the epipolar geometry between two images. In simpler terms, it explains how a point in one image limits where the matching point can be located in the next image. This step is important because it gives a geometric relationship between two camera views.

I used RANSAC when estimating the fundamental matrix. This was important because even after using the ratio test, some incorrect matches can still remain. RANSAC helps by estimating the fundamental matrix while ignoring outliers. It repeatedly tests different sets of matches and chooses the model that fits the largest number of good correspondences. This makes the result more stable and less affected by bad matches.

After finding the fundamental matrix, I computed the essential matrix. The essential matrix was calculated using the equation:

$$E = K.T * F * K$$

In this equation, E is the essential matrix, F is the fundamental matrix, and K is the intrinsic camera matrix. This step is where the camera calibration parameters are used. The fundamental matrix works with pixel coordinates, but the essential matrix works with normalized camera coordinates. The essential matrix is what allows us to recover the camera's actual relative motion between two frames.

Then, I used cv2.recoverPose to recover the rotation and translation parameters from the essential matrix. The rotation matrix R describes how the camera's orientation changed between two frames. The translation vector t describes the direction of the camera's movement. The essential matrix can produce multiple possible solutions, but cv2.recoverPose chooses the physically valid one by checking which solution places the reconstructed points in front of the cameras. This is called the depth positivity constraint.

After testing these steps on one pair of images, I applied the same process to the full image sequence. For every pair of consecutive frames, I loaded the two images, detected SIFT keypoints, computed descriptors, matched the descriptors with FLANN, filtered the matches using the ratio test, estimated the fundamental matrix with RANSAC, computed the essential matrix, and recovered the rotation and translation. I stored the rotations and translations in separate lists so I could use them later to reconstruct the full camera trajectory. This matched the project requirement to estimate rotation and translation between successive frames and store them for the trajectory reconstruction step.

The final step was reconstructing the camera path. I started the first camera center at the origin. Then, using the rotation and translation values from each pair of frames, I estimated the next camera position. By repeating this process through the sequence, I was able to build a set of camera centers over time. I then plotted the x and z coordinates using Matplotlib to create a top-down view of the trajectory.

The final map was constructed successfully and portrays a realistic camera motion path. The plotted trajectory shows continuous movement instead of random jumps, which suggests that the motion estimation worked reasonably well. The path also shows forward movement and changes in direction, which is what I would expect from a camera mounted on a moving car. Overall, the result gives a believable visual representation of the camera's movement through the driving sequence.

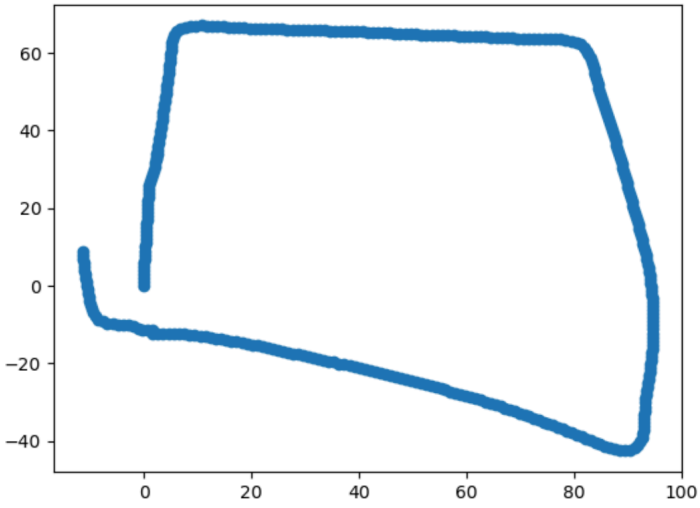

Figure 2. Reconstructed camera trajectory projected onto the x-z plane.

At the same time, the result should not be treated as a perfect GPS-level reconstruction. Since this project uses monocular visual odometry, the translation recovered from the essential matrix does not provide an exact real-world scale. This means the plot shows the general shape and direction of the camera's motion, but not exact real-world distances. Another limitation is drift. Small errors in feature matching, matrix estimation, or pose recovery can accumulate over time and slightly distort the final trajectory. Even with these limitations, the final result still shows that the pipeline successfully estimated and visualized the camera's motion.

Overall, this project helped me understand how the different parts of visual odometry fit together. I learned how camera intrinsics are used in motion estimation, how keypoints can be detected and matched between frames, how the fundamental and essential matrices describe the relationship between camera views, and how rotation and translation can be recovered from the essential matrix. Most importantly, I learned how frame-to-frame motion estimates can be accumulated to reconstruct a full camera trajectory. The final result shows that the camera path was successfully reconstructed and that the map portrays a realistic camera motion path.

I would like to give credit to my peers on Piazza for their helpful ideas, explanations, and tutorials that supported my understanding of the project.